

Figure 1: Containers vs virtual machine architecture (Image source: [googleimages.com](#))

management issues. Overall, containers help us take the DevOps culture a step ahead by increasing the portability of an application. They help to increase the storage and CPU efficiency as well.

What is containerisation?

Containerisation is nothing but the process of abstracting away all the differences in operating system distributions and their underlying infrastructure by encapsulating discrete components of application logic, including the application platform and its dependencies, with the help of lightweight containers. It's actually a virtualisation method for deploying different distributed applications without actually launching an entire virtual machine for each application—multiple isolated systems run on a single control host and they access a single kernel. The containers used for containerisation hold different components such as environment variables, files and libraries, which are necessary to run the desired software. Since different resources are shared in this way, containers that place less strain on the overall available resources can be created. For instance, if a specific variation from any standard image is desired, one can create a container that holds only the new library.

Why containerisation?

1. Since containerisation does not incur any performance overhead as in the case of virtualisation, there is a high possibility that many containers can be supported on the same infrastructure.
2. It is because of containerisation that any application container can easily run on any system or in any cloud without any code changes. In the case of containerisation, there are no guest OS environment variables or any library dependencies to manage.
3. Containerisation is also required when we have to rely on the behaviour of a specific version of the SSL library for any application to run, and all of a sudden an updated version of it automatically gets installed. Containerisation lets the application run without any exception in such cases.
4. It also plays a significant role when there are differences in the network topology, or even in the security or storage

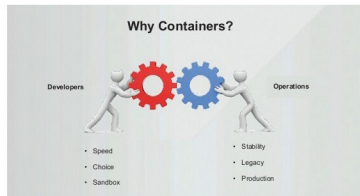


Figure 2: Containerisation for the developer and operations team (Image source: [googleimages.com](#))

- policies of the two platforms on which software needs to run. Containerisation makes it possible to run the containerised software on any such platform.
5. A server running three different containerised applications actually runs on a single OS, and each of its containers shares the OS kernel with other containers. Shared parts of the OS are in read-only mode, while each container has its own way to access its peer containers for writing. This proves that containers are much more lightweight and also use fewer resources than the virtual machines, and hence are preferred.
 6. A container used for containerisation may be only of megabyte-size, whereas a virtual machine with its own OS may be several gigabytes in size. Hence, a single server can host more containers than virtual machines, giving it another edge over VMs.
 7. Containerised applications do not have any boot time. They can be started instantaneously. This means containers can be instantiated just when required, and can also disappear when no longer required, freeing up the resources on their hosts.
 8. Containers are always designed in such a way that they run on everything, right from physical computers to virtual machines, bare-metal servers, different OpenStack cloud clusters, etc.
 9. Containerisation also enables different developers to fully 'own' the configuration and set-up of their application's runtime environment.
 10. Containerisation leads to greater modularity. Instead of running an entire complex application using a single container, the application can also be split into different modules such as the application front-end, the database and so on. This is called the microservices approach. Applications built in such a way are easier to manage as each of their modules is relatively simple. Also, different changes can be made to different modules without the hassle of rebuilding the entire application.
 11. Containerisation supports a more unified DevOps culture. All developers have to handle applications and their frameworks, while the IT operations team is concerned